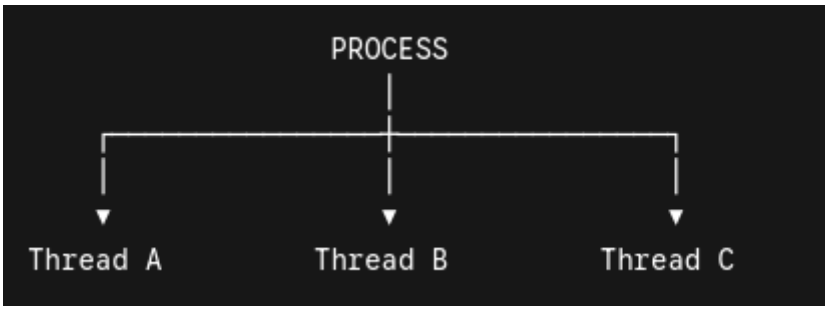
[Dat Nguyen] Threads & Synchronization

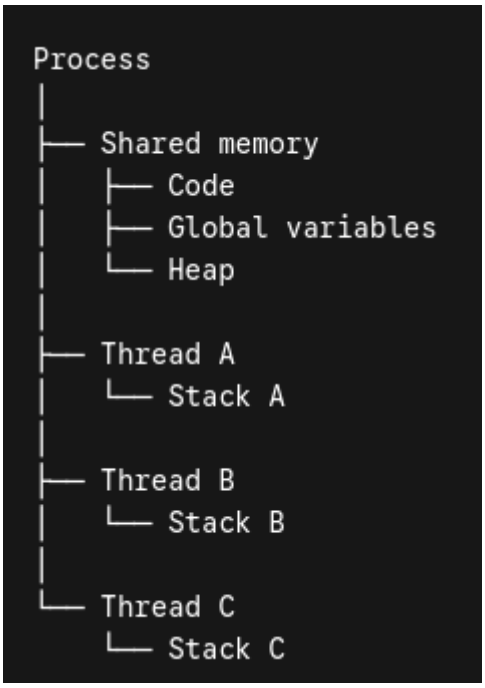
1. Thread Concepts, Thread Identification, Thread Creation, Thread Termination

1.1 Thread Concepts

- A thread is a mechanism that allows an application to run multiple tasks at the same time. A single process can contain multiple threads running in parallel.



- Threads inside the same process share many process resources, but each thread has its own stack and execution state.



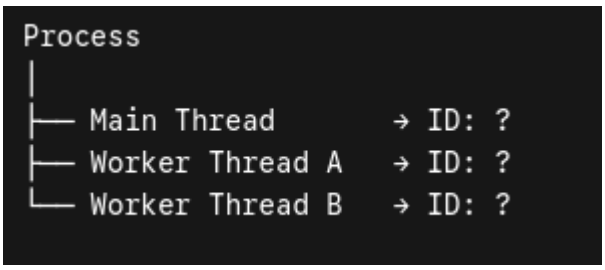
- Imagine a Process as a company, and Threads as its employees:

  ```
  Employees share the same office, printer, storage room, and company budget (Heap, Code, Data, Files).

  However, each employee has their own personal notebook (Stack) to write down their own daily tasks. Employees do not share their notebooks with each other.
  ```

- The CPU can schedule threads to run by switching quickly between them or running them truly at the same time on multiple CPU cores.

1.2 Thread Identification



- Data type for Thread ID: pthread_t
- Each thread in a process has a unique Thread ID.
- Get current thread ID: pthread_self()
- Compare two Thread IDs: pthread_equal(t1, t2) — Used to check if two pthread_t variables represent the same thread.
- Main thread waits for a worker thread to finish: pthread_join()

1.3 Thread Creation

When a program starts, the initial process has only one thread, called the initial thread or main thread.

```
pthread_create(pthread_t *thread, const pthread_attr_t *attr,
           void *(*start_routine)(void *), void *arg);
```

Parameter	Meaning	Common Choice
`thread`	Pointer to store the ID of the newly created Thread	`&tid`
`attr`	Attribute configuration (Stack size, priority...)	`NULL` (Default)
`start_routine`	Function that the Thread will run right after creation	Function name (e.g., `my_task`)
`arg`	Data passed into the function	`NULL` or pointer `(void*)&data`

Main thread wait for thread ended: pthread_join(thread, NULL);

```
void *worker1(void *arg)
{
   char *msg1 = (char *)arg;
   printf("Worker1 received: %s\n", msg1);
   return NULL;
}

void *worker2(void *arg)
{
   char *msg2 = (char *)arg;
   printf("Worker2 received: %s\n", msg2);
   return NULL;
}
int main()
{
   pthread_t thread1, thread2;
   char *message1 = "Xin woker1 from Main!";
   char *message2 = "Xin woker2 from Main!";

   pthread_create(&thread1, NULL, worker1, (void *)message1);
   pthread_create(&thread2, NULL, worker2, (void *)message2);

   pthread_join(thread1, NULL);
   pthread_join(thread2, NULL);
```

```
    return 0;
}
```

Output:

```
Worker1 received: Xin woker1 from Main!
Worker2 received: Xin woker2 from Main!
```

1.4 Thread Termination

There are 3 main ways a thread terminates:

- return from the thread's start routine.
- Calling pthread_exit().
- The process terminates -> all threads in the process are terminated immediately.

return vs pthread_exit()

- return in main(): The process exits, and all running worker threads are terminated immediately.

```
Process
│
├── Main ──> return 0 ──> Process END
├── Thread A ──────────── X
└── Thread B ──────────── X
```

- pthread_exit() in main(): The main thread finishes, but worker threads continue running until they complete.

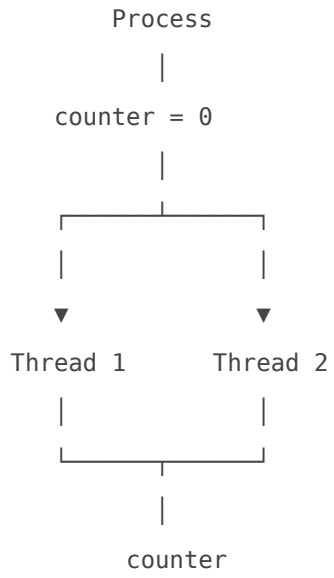
```
Process
│
├── Main ──> pthread_exit() ──> END
│
├── Thread A ──────────────────> RUNNING
│
└── Thread B ──────────────────> RUNNING
```

Getting Thread Return Value with pthread_join()

- pthread_join(thread, NULL); — Wait for the thread to finish, ignore return value.
- pthread_join(thread, &ret); — Wait for the thread to finish and capture its return value.

2. Why concurrency is hard: race conditions and the need for mutual exclusion

2.1. Shared data



=> Creates a Race Condition

A Race Condition occurs when the program's output depends on the exact execution timing or scheduling order of its threads.

```
#define NUM 1000000
int counter = 0;
void *worker(void *arg)
{
   for (int i = 0; i < NUM; i++)
   {
       counter++;
   }
   return NULL;
}
int main()
{
   pthread_t thread1;
   pthread_t thread2;
```

```
    pthread_create(&thread1, NULL, worker, NULL);
    pthread_create(&thread2, NULL, worker, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("Expected: %d\n", NUM * 2);
    printf("Actual:   %d\n", counter);
    return 0;
}
```

Output:

```
Expected: 2000000
Actual:   1354200
```

When two threads execute the `worker` function concurrently, these steps interleave with each other, causing multiple increments to be overwritten and lost -> **Critical Section is required**.

2.2 Mutual Exclusion

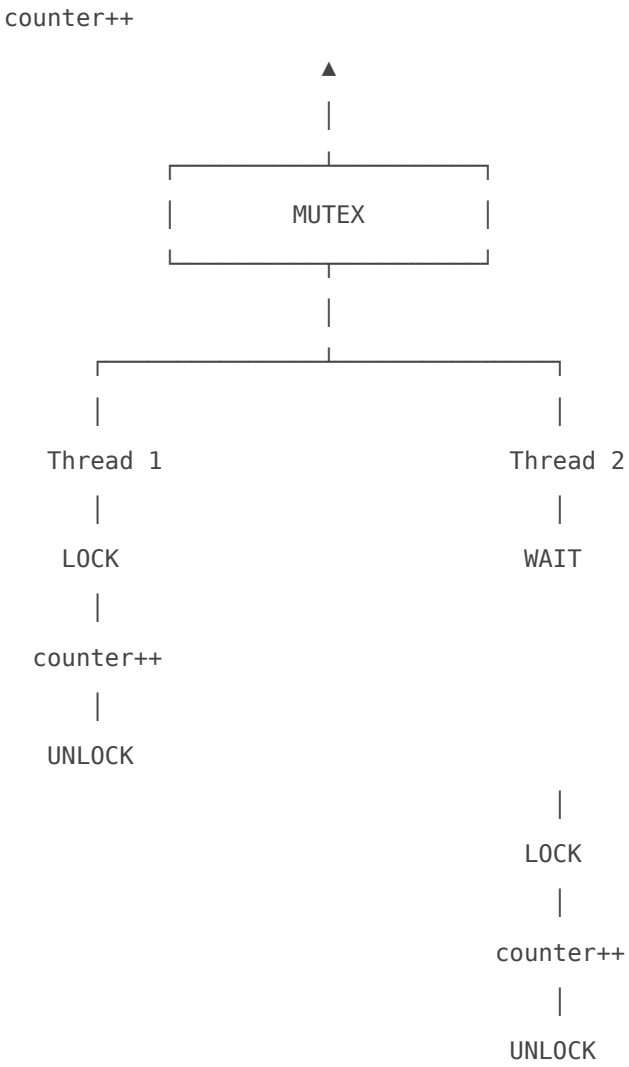
A Critical Section is a block of code that accesses a shared resource and must be protected to prevent multiple threads from accessing it concurrently in an unsafe manner.

```
Two threads access shared data concurrently
                   ↓
            Race Condition
                   ↓
        Mutual Exclusion needed
                   ↓
                Use Mutex
                   ↓
Only one thread enters Critical Section at a time
```

3. Mutex Locks & Critical Sections

3.1 Mutex Lock

A Mutex works like a single key to a room. Only the thread that holds the key is allowed to enter the protected area (Critical Section). All other threads must wait until the key is returned.



3.2 Critical section

A Critical Section is a sequence of code in a multi-threaded or multi-process program that accesses shared resources (such as global variables or files).

```
pthread_mutex_lock(&mutex);
counter++;
pthread_mutex_unlock(&mutex);
```

3.3 pthread_mutex_t, Lock, and Unlock

Initialization: pthread_mutex_init(&mutex, NULL);

Destruction: pthread_mutex_destroy(&mutex);

Locking: pthread_mutex_lock(&mutex);

Unlocking: pthread_mutex_unlock(&mutex);

3.4 Fixing the Race Condition Program

```
#define NUM 1000000
int counter = 0;

void *worker(void *arg)
{
   for (int i = 0; i < NUM_INCREMENTS; i++)
   {
       pthread_mutex_lock(&mutex);
       counter++;
       pthread_mutex_unlock(&mutex);
   }
   return NULL;
}
```

```
pthread_mutex_lock()
        ↓
   Acquire mutex
        ↓
Enter critical section
        ↓
pthread_mutex_unlock()
        ↓
  Release mutex
```

Output:

```
Expected: 2000000
Actual:   2000000
```

Note: Mutex does **not** guarantee that Thread 1 will run before Thread 2. It only guarantees that **no two threads** enter the critical section at the same time.

3.5 Locking the Entire for Loop

Code:

```
pthread_mutex_lock(&mutex);

for (int i = 0; i < NUM_INCREMENTS; i++)
```

```
{
    counter++;
}

pthread_mutex_unlock(&mutex);
```

Advantages (High Locking Performance)

```
Minimal overhead: Only 1 Lock and 1 Unlock operation per thread.

Fast execution speed: Eliminates the overhead of continuously calling Mutex lock/unlock inside
the loop.
```

Disadvantages (Loss of Concurrency)

```
Converts multi-threading to sequential execution: Holding the lock for the entire loop forces
Thread B to
wait until Thread A finishes. The threads end up running sequentially rather than in parallel.
```

Practical Lesson

Multi-threaded programming requires finding a balance between:

```
Shrinking the Critical Section: Keeps threads from waiting on each other for extended periods.

Reducing Lock/Unlock Frequency: Prevents wasting CPU cycles on repeated synchronization calls.
```

4. Reader-Writer Locks

4.1 Concept

Standard Mutex locks allow only one thread inside the protected region at a time, even when all threads are only reading shared data.

```
Shared data
    │
    ├── Thread A → READ
    ├── Thread B → READ
    ├── Thread C → READ
    └── Thread D → WRITE
```

If Threads A, B, and C only perform read operations, using a Mutex forces them to execute sequentially and wait for each other. Allowing concurrent read operations while ensuring data safety requires a Reader-Writer Lock (RWLock).

4.2 API

Operation	Function Call	Description
Declaration	`pthread_rwlock_t rwlock;`	Declares a Reader-Writer lock variable
Initialization	`pthread_rwlock_init(&rwlock, NULL);`	Initializes the lock
Reader Lock	`pthread_rwlock_rdlock(&rwlock);`	Acquires a shared lock for reading
Writer Lock	`pthread_rwlock_wrlock(&rwlock);`	Acquires an exclusive lock for writing
Unlock	`pthread_rwlock_unlock(&rwlock);`	Releases either a read or write lock
Destruction	`pthread_rwlock_destroy(&rwlock);`	Destroys the lock when no longer needed

4.3 pthread_rwlock_t Behavior

```
Operations:

    Reader: Reads data without modifying it.

    Writer: Modifies or updates data.
```

Multiple Readers (Shared Access): Multiple reader threads can access shared data simultaneously.

```
Reader 1 ──┐
Reader 2 ──┼──> Shared Data
Reader 3 ──┘
```

Writer (Exclusive Access): A writer requires exclusive access. All readers and other writers must wait while a write operation is active.

```
Writer
        │
        ▼
   Shared Data
        ▲
        │
All Readers must WAIT
```

4.4 Code

```
int shared_data = 100;

pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;

void *reader(void *arg)
{
    int id = *(int *)arg;

    pthread_rwlock_rdlock(&rwlock);

    printf("Reader %d: data = %d\n", id, shared_data);

    sleep(1);

    pthread_rwlock_unlock(&rwlock);

    return NULL;
}

void *writer(void *arg)
{
    pthread_rwlock_wrlock(&rwlock);

    printf("Writer: modifying data...\n");

    shared_data += 10;

    sleep(1);

    printf("Writer: data = %d\n", shared_data);

    pthread_rwlock_unlock(&rwlock);

    return NULL;
}

int main()
{
```

```
    pthread_t readers[3];
    pthread_t writer_thread;

    int ids[3] = {1, 2, 3};

    pthread_create(&readers[0], NULL, reader, &ids[0]);
    pthread_create(&readers[1], NULL, reader, &ids[1]);
    pthread_create(&readers[2], NULL, reader, &ids[2]);

    pthread_create(&writer_thread, NULL, writer, NULL);

    for (int i = 0; i < 3; i++)
    {
        pthread_join(readers[i], NULL);
    }

    pthread_join(writer_thread, NULL);

    pthread_rwlock_destroy(&rwlock);

    return 0;
}
```

Output:

```
Reader 1: data = 100
Reader 2: data = 100
Reader 3: data = 100
Writer: modifying data...
Writer: data = 110
```

5. Reentrancy and Thread-Specific Data

5.1 Reentrancy

A function is **reentrant** if multiple execution instances of the function can overlap in time without corrupting data.

Overlapping executions occur when:

- A function is partially executed and then paused (due to an interrupt, context switch).
- A new execution instance of the same function starts and completes.
- The original execution instance resumes and completes correctly, without data corruption or invalid state.

A function easily achieves reentrancy by following these conditions:

```
Avoid Global/Static Variables: Uses local (stack) variables instead of global or static storage.

Avoid Returning Pointers to Static Shared Buffers: Returns data via caller-allocated buffers or value pass-throughs.

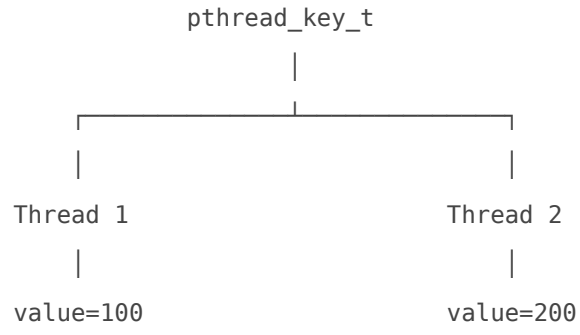
Avoid Shared State Dependencies: Ensures execution depends solely on input parameters.

Call Only Reentrant Functions: Invokes exclusively other functions that are themselves reentrant.
```

5.2 Thread-Specific Data (TSD)

Thread-Specific Data creates a global variable interface where each thread maintains its own distinct value.

```
For example, given a single global variable value, access by Thread 1 and Thread 2 maintains independent storage space (value = 100 for Thread 1 and value = 200 for Thread 2) without declaring explicit separate variables value1 and value2.
```



Both threads access the exact same key, but each reads and writes its own private copy of data.

5.3 API

Operation	Function Call	Description
Create Key	`pthread_key_create(&key, destructor)`	Allocates a key for TSD and assigns an optional cleanup function

Operation	Function Call	Description
Set Specific	`pthread_setspecific(key, pointer)`	Associates a thread-private pointer with the key
Get Specific	`pthread_getspecific(key)`	Retrieves the thread-private pointer associated with the key
Delete Key	`pthread_key_delete(key)`	Destroys the key (does not invoke the destructor)

example:

```
// 1. Declare a single global key for all threads
pthread_key_t key;

void* worker(void* arg) {
    char* name = (char*)arg;

    // 2. Each thread saves its own name using the same key
    pthread_setspecific(key, name);

    // Simulate work to test data independence between threads
    usleep(100000);

    // 3. Read data from the key
    char* stored_name = (char*)pthread_getspecific(key);

    // 4. Print the result
    printf("Thread %ld: %s\n", (long)pthread_self(), stored_name);

    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    // Private data for each thread
    char* name1 = "Alice";
    char* name2 = "Bob";
    char* name3 = "Charlie";
```

```
    // Initialize the shared key (No destructor needed since we use string constants, not
dynamic memory)
    pthread_key_create(&key, NULL);

    // Create 3 threads and pass the corresponding data
    pthread_create(&t1, NULL, worker, name1);
    pthread_create(&t2, NULL, worker, name2);
    pthread_create(&t3, NULL, worker, name3);

    // Wait for the threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    // Delete the key after finishing
    pthread_key_delete(key);

    return 0;
}
}
```

Output:

```
Thread 124204766721728: Alice
Thread 124204758329024: Bob
Thread 124204749936320: Charlie
```

6 Threads and Signals, fork(), and I/O

6.1 Thread and Signal

A Signal is like an asynchronous notification sent to a process or thread.

Signal	Trigger	Description
SIGINT	`Ctrl + C`	Interrupt signal from keyboard
SIGTERM	`kill <PID>`	Safe request to terminate process
SIGKILL	`kill -9 <PID>`	Kill process immediately (cannot be blocked)
SIGALRM	`alarm()`	Timer signal

Signal	Trigger	Description
SIGUSR1	User-defined	Custom signal defined by programmer

- **Process-directed signal:**

Example:

```
Process
│
├── Main Thread
│
├── Worker 1
│
└── Worker 2
```

=> Which thread will receive the signal?

The kernel picks a suitable thread to deliver the signal based on the signal mask/status of the threads.

- **Thread-directed signal: Send a signal to a specific thread using `pthread_kill():`**

code:

```
// REQUIRED: volatile sig_atomic_t ensures atomicity and prevents compiler optimization
static volatile sig_atomic_t got_sigusr1 = 0;

// Signal Handler: Very brief, ONLY sets the flag and exits immediately
void sig_handler(int sig)
{
    if (sig == SIGUSR1) {
        got_sigusr1 = 1; // Set a flag
    }
}

void *worker_thread(void *arg)
{
    printf("[Worker] Thread running and waiting for signal...\n");

    while (1)
    {
        // 1. Check the flag in normal execution flow
```

```
        if (got_sigusr1) {
            got_sigusr1 = 0; // Reset flag for next time

            // 2. Safely perform heavy/complex work here!
            printf("--> [Worker Normal Code] Flag set! Processing heavy task\n");
            sleep(1);
            printf("--> [Worker Normal Code] Heavy task finished!\n");
        }

        // Normal periodic work
        printf("[Worker] Doing normal work...\n");
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_t worker;

    // Register Process-level Handler
    signal(SIGUSR1, sig_handler);

    // Create Worker Thread
    pthread_create(&worker, NULL, worker_thread, NULL);

    // Wait 2 seconds for Worker to initialize
    sleep(2);

    // Send SIGUSR1 to Worker
    printf("\n[Main] Sending SIGUSR1 to Worker...\n");
    pthread_kill(worker, SIGUSR1);

    // Keep Main running for a few seconds to observe Worker
    sleep(4);

    pthread_cancel(worker);
    pthread_join(worker, NULL);
    printf("[Main] Program finished.\n");
```

```
    return 0;
}
```

Output:

```
[Worker] Thread running and waiting for signal...
[Worker] Doing normal work...
[Worker] Doing normal work...

[Main] Sending SIGUSR1 to Worker...
--> [Worker Normal Code] Flag set! Processing heavy task
--> [Worker Normal Code] Heavy task finished!
[Worker] Doing normal work...
[Worker] Doing normal work...
[Worker] Doing normal work...
[Main] Program finished.
```

- Signal mask: A "filter" that lets a thread decide: Which signals are allowed to interrupt it, and which signals must be blocked.

```
Process
│
├── T1 → signal mask A  (Allows receiving Signal X, Y)
├── T2 → signal mask B  (Blocks all signals to do heavy work)
└── T3 → signal mask C  (Only receives Signal Z)
```

Action	C Syntax	Signal Set State (`set`)
Create signal set	`sigset_t set;`	Uninitialized
Clear set	`sigemptyset(&set);`	`set = {}`
Add signal	`sigaddset(&set, SIGUSR1);`	`set = { SIGUSR1 }`
Block signals	`pthread_sigmask(SIG_BLOCK, &set, NULL);`	Apply `set` to Thread Mask

code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <pthread.h>

void handler(int sig) {
```

```
  printf("   ---> [HANDLER] Signal %d handled by Thread TID: %lu\n", sig, (unsigned
long)pthread_self());
}

// T1: Fully open (Receives both SIGUSR1 and SIGUSR2)
void *thread1_func(void *arg) {
  printf("[T1] thread_id = %lu Running with FULLY OPEN Mask (Ready to receive SIGUSR1 &
SIGUSR2)\n", (unsigned long)pthread_self());
  while (1) sleep(1);
  return NULL;
}

// T2: Blocks all signals (Heavy task)
void *thread2_func(void *arg) {
  sigset_t set;
  sigfillset(&set); // Add all signals to set
  pthread_sigmask(SIG_BLOCK, &set, NULL); // Block all!

  printf("[T2] thread_id = %lu Running with BLOCKED ALL Mask (Immune to all signals)\n",
(unsigned long)pthread_self());
  while (1) sleep(1);
  return NULL;
}

// T3: Only receives SIGUSR2 (Blocks SIGUSR1)
void *thread3_func(void *arg) {
  sigset_t set;
  sigemptyset(&set);
  sigaddset(&set, SIGUSR1); // Block SIGUSR1 only
  pthread_sigmask(SIG_BLOCK, &set, NULL);


  printf("[T3] thread_id = %lu Running with BLOCK SIGUSR1 Mask (Only receives SIGUSR2)\n",
(unsigned long)pthread_self());
  while (1) sleep(1);
  return NULL;
}

int main() {
  pthread_t t1, t2, t3;
```

```
  signal(SIGUSR1, handler);
  signal(SIGUSR2, handler);

  pthread_create(&t1, NULL, thread1_func, NULL);
  pthread_create(&t2, NULL, thread2_func, NULL);
  pthread_create(&t3, NULL, thread3_func, NULL);

  sleep(1); // Wait for 3 threads to set up masks: High risk of Race Condition if signals are sent before masks are fully initialized.


  // TEST 1: Send SIGUSR1 to T2 (Blocks all)
  printf("\n[Main] Sending SIGUSR1 to T2 (Blocks all)...");
  pthread_kill(t2, SIGUSR1);
  sleep(1); // Nothing happens!

  // TEST 2: Send SIGUSR1 to T3 (Blocks SIGUSR1)
  printf("\n[Main] Sending SIGUSR1 to T3 (Blocks SIGUSR1)...");
  pthread_kill(t3, SIGUSR1);
  sleep(1); // Nothing happens!

  // TEST 3: Send SIGUSR2 to T3 (Allows SIGUSR2)
  printf("\n[Main] Sending SIGUSR2 to T3 (Allows SIGUSR2)...");
  pthread_kill(t3, SIGUSR2);
  sleep(1); // Handler runs!

  // TEST 4: Send SIGUSR1 to T1 (Fully open)
  printf("\n[Main] Sending SIGUSR1 to T1 (Fully open)...");
  pthread_kill(t1, SIGUSR1);
  sleep(1); // Handler runs!

  return 0;
}
```

Output:

```
[T1] thread_id = 138050373940928 Running with FULLY OPEN Mask (Ready to receive SIGUSR1 & SIGUSR2)
[T2] thread_id = 138050365548224 Running with BLOCKED ALL Mask (Immune to all signals)
[T3] thread_id = 138050357155520 Running with BLOCK SIGUSR1 Mask (Only receives SIGUSR2)
```

```
[Main] Sending SIGUSR1 to T2 (Blocks all)...
[Main] Sending SIGUSR1 to T3 (Blocks SIGUSR1)...
[Main] Sending SIGUSR2 to T3 (Allows SIGUSR2)...   ---> [HANDLER] Signal 12 handled by Thread TID: 138050357155520

[Main] Sending SIGUSR1 to T1 (Fully open)...   ---> [HANDLER] Signal 10 handled by Thread TID: 138050373940928
```

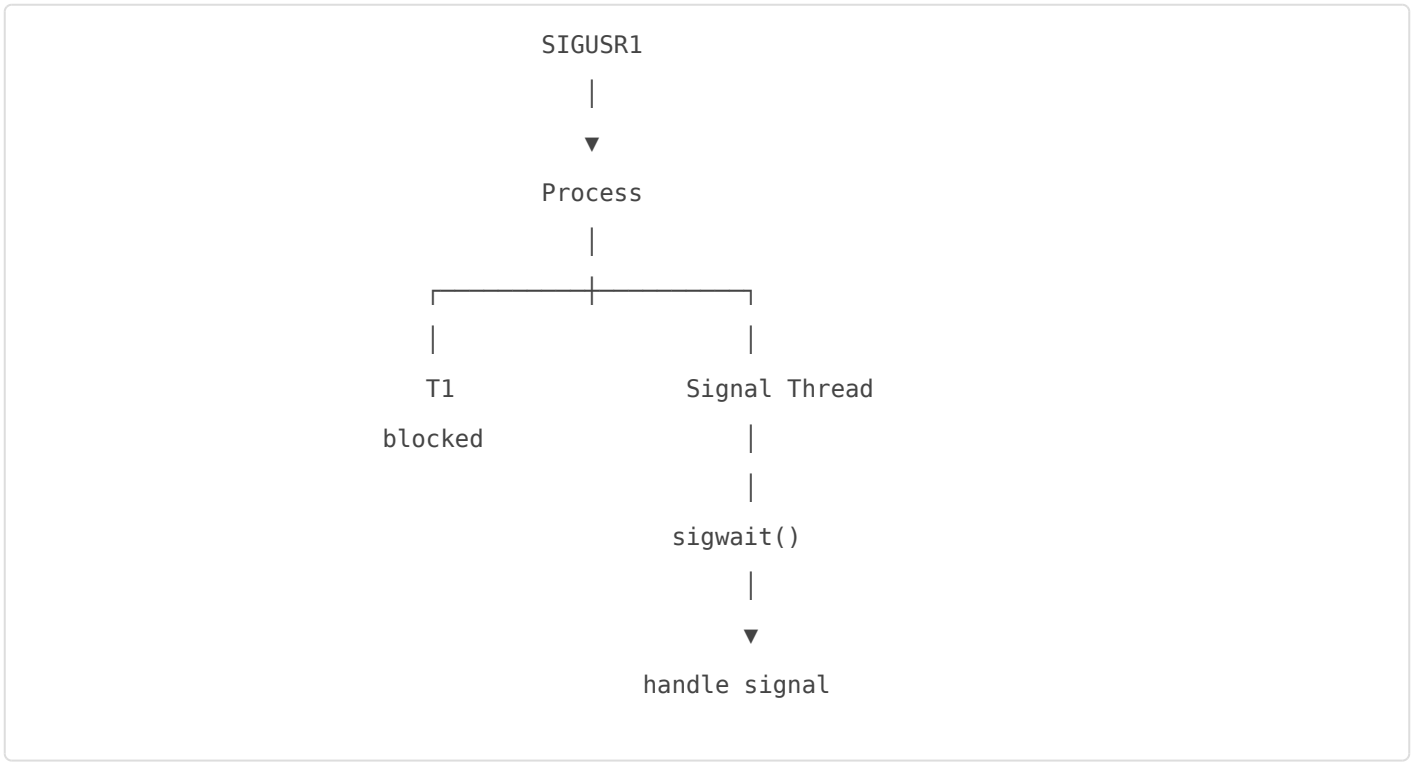
Summary:

```
Summary:
                                SIGNAL
                                  |
                  ┌───────────────┴──────────────┐
                  |                              |
      Process-directed                  Thread-directed
             |                                 |
         kill()                          pthread_kill()
            |                                 |
            ▼                                 ▼
         Process                           Thread
            |
            ▼
     Kernel chooses a
    suitable thread to deliver

          Thread
            |
            └── Signal Mask
                     |
                     ├── SIGUSR1 blocked
                     ├── SIGTERM blocked
                     └── …
```

- **sigwait()**: While standard signal handling makes a thread **passive** by interrupting it unpredictably, `sigwait()` allows the thread to be **active** by blocking signals and controlling exactly when to wait for and process them.

Flow:



code

```
#include <stdio.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>

// Worker 1: Prints "Worker 1 running" every 2 seconds
void *worker1_func(void *arg) {
    while (1) {
        printf("Worker 1 running\n");
        sleep(2);
    }
    return NULL;
}

// Signal Thread: Waits for SIGUSR1 using sigwait() in an infinite loop
void *signal_thread_func(void *arg) {
    sigset_t *set = (sigset_t *)arg;
    int sig;

    while (1) {
        // Actively wait for a signal from the Pending Queue
        if (sigwait(set, &sig) == 0) {
```

```
            if (sig == SIGUSR1) {
                printf("SIGUSR1 received\n");
            }
        }
    }
    return NULL;
}

int main() {
    pthread_t w1, sig_tid;
    sigset_t set;

    // 1. Initialize sigset_t and add SIGUSR1
    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);

    // 2. Block SIGUSR1 in Main Thread (Worker threads will inherit this mask)
    pthread_sigmask(SIG_BLOCK, &set, NULL);

    // 3. Create Worker thread
    pthread_create(&w1, NULL, worker1_func, NULL);

    // 4. Create Signal Thread (Pass &set as argument)
    pthread_create(&sig_tid, NULL, signal_thread_func, &set);

    // 5. Wait for Signal Thread
    pthread_join(sig_tid, NULL);

    return 0;
}
```

Note notepthread_sigmask() is called before creating Worker threads because:

```
Mask Inheritance Mechanism: All worker threads created afterwards automatically inherit this "shield" blocking the signal.

Preventing Race Conditions: If a thread were created with an UNBLOCKED mask, the Kernel could immediately deliver a signal to it before its mask could be set.
```

Output:

```
Open 2 terminals

    Terminal 1: Used to observe the running program

    Terminal 2: Used to send signals from the kernel
    # Find the PID of the current process
    ps aux | grep app

    # Send the signal to the PID
    kill -SIGUSR1 <PID>

  Worker 1 running
  Worker 1 running
  Worker 1 running
  SIGUSR1 received
  Worker 1 running
  SIGUSR1 received
  Worker 1 running
  SIGUSR1 received
  Worker 1 running
```

6.2 Thread and fork()

```
Process
│
├── Main Thread
├── Worker 1
└── Worker 2
```

After fork()

```
Parent                          Child
│                                 │
├── Main                          └── Main
├── Worker 1
└── Worker 2
```

fork() in a multithreaded process copies ONLY the thread that called fork() into the child process.

```
Thread Behavior: The child process becomes single-threaded, inheriting only the execution context of the specific thread that invoked fork(). All other threads from the parent process simply vanish in the child.

Potential Risk: If any of those lost threads held a mutex lock at the exact moment fork() was called, that mutex will remain locked forever in the child process, leading to potential deadlocks.
```

code: Potential risk

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg)
{
    pthread_mutex_lock(&mutex);

    printf("Worker: mutex locked\n");

    sleep(10);

    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main()
{
    pthread_t thread;

    pthread_create(&thread, NULL, worker, NULL);

    sleep(1);

    printf("Main: fork()\n");

    pid_t pid = fork();
```

```
    if (pid == 0)
    {
        printf("Child: trying to lock mutex...\n");

        pthread_mutex_lock(&mutex);

        printf("Child: acquired mutex\n");

        pthread_mutex_unlock(&mutex);
    }
    else
    {
        pthread_join(thread, NULL);
    }

    return 0;
}
```

explain: **Output:**

```
Output:
Worker: mutex locked
Main: fork()
Child: trying to lock mutex…
```

explain:

- Worker locks the mutex & sleeps for 10 seconds: This changes the mutex state in RAM memory to LOCKED.
- Main calls fork() while Worker is sleeping: The Child Process copies a snapshot of the Parent's RAM, inheriting the mutex in its LOCKED state.
- fork() only copies the calling thread (Main Thread): This causes the Worker thread to completely disappear in the Child Process.
- Child Process calls pthread_mutex_lock(): It sees that the mutex is already locked, so it waits. But since no Worker thread exists in the Child Process to unlock it -> the Child Process is stuck forever (Deadlock).

Fix:

pthread_atfork() automatically locks all mutexes BEFORE fork(), and unlocks them in BOTH the parent and child processes AFTER fork() finishes.

```
pthread_atfork(
    prepare,                    : run before fork()
    parent,                     : run in parent after fork
    child                       : run in child after fork
);
```

code fix:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void prepare(void)
{
   pthread_mutex_lock(&mutex);
}

void parent(void)
{
   pthread_mutex_unlock(&mutex);
}

void child(void)
{
   pthread_mutex_unlock(&mutex);
}

void *worker(void *arg)
{
   pthread_mutex_lock(&mutex);

   printf("Worker: mutex locked\n");

   sleep(10);

   pthread_mutex_unlock(&mutex);

   return NULL;
```

```
}

int main()
{
   pthread_atfork(prepare, parent, child);

   pthread_t thread;

   pthread_create(&thread, NULL, worker, NULL);

   sleep(1);

   printf("Main: fork()\n");

   pid_t pid = fork();

   if (pid == 0)
   {
       printf("Child: trying to lock mutex...\n");

       pthread_mutex_lock(&mutex);

       printf("Child: acquired mutex\n");

       pthread_mutex_unlock(&mutex);
   }
   else
   {
       pthread_join(thread, NULL);
   }

   return 0;
}
```

Output

```
output:
Worker: mutex locked
Main: fork()
Child: trying to lock mutex...
```

```
Child: acquired mutex
```

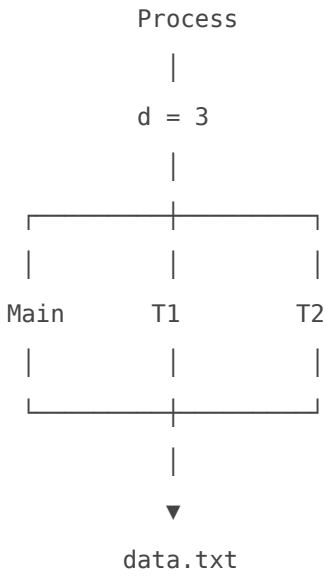
explain:

- Main calls fork(), the system automatically runs the prepare() function. prepare() calls pthread_mutex_lock(&mutex). Since the worker is holding the lock (sleeping for 10s), main actively waits for about 9 seconds until the worker wakes up and releases the lock.
- When the worker unlocks it $\rightarrow$ prepare() gets the lock: The system takes a snapshot of the RAM to create the Child Process.
- After the Child is created, the child() function runs in the Child Process and calls pthread_mutex_unlock(&mutex) -> releasing the lock in the Child.
- The Child Process continues running, calls pthread_mutex_lock(), and successfully prints "Child: acquired mutex"!

6.3 Thread and I/O

```
Process
│
├── Main
├── Worker 1
└── Worker 2
```

int fd = open("data.txt", O_RDONLY);



code:

```
#include <stdio.h>
#include <fcntl.h>
```

```c
#include <unistd.h>
#include <pthread.h>

int fd;

void *worker1(void *arg)
{
    char buffer[6] = {0};

    read(fd, buffer, 5);

    printf("Worker 1: %s\n", buffer);

    return NULL;
}

void *worker2(void *arg)
{
    char buffer[6] = {0};

    read(fd, buffer, 5);

    printf("Worker 2: %s\n", buffer);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    fd = open("data.txt", O_RDONLY);

    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    close(fd);
```

```
    return 0;
}
```

output: data.txt = "HelloWorld"

```
Woker1 1: Hello
Worker 2: World
```

explain:

- Worker 1 and Worker 2 share the same file offset pointer.
- Offset = 0: Worker 1 reads 5 bytes => automatically advances the offset to 5.
- Worker 2 continues reading starting from Offset = 5.

Race condition: If two threads call a sequence of commands to move the file pointer and write data at the same time.

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <fcntl.h>

int fd;

void *worker1(void *arg)
{
    // Step 1: T1 wants to move the pointer to position A (Offset 0)
    lseek(fd, 0, SEEK_SET);

    // Simulate a context switch immediately after seeking
    usleep(100);

    // Step 2: T1 writes data (Expecting to write at Offset 0)
    write(fd, "HELLO", 5);
    return NULL;
}
```

```
void *worker2(void *arg)
{
    // Step 1: T2 jumps in and moves the pointer to position B (Offset 10)
    lseek(fd, 10, SEEK_SET);

    // Step 2: T2 writes data
    write(fd, "WORLD", 5);
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    // Open or create the data.txt file with Read/Write permissions
    fd = open("data.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Failed to open file");
        return 1;
    }

    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    close(fd);
    return 0;
}
```

output

```
WORLDHELLO
```

explain:

- T1 calls lseek(0): Moves the file pointer to position 0, then goes to sleep with usleep.
- T2 jumps in and calls lseek(10): Changes the shared file pointer to position 10.
- T2 calls write("WORLD"): Writes at offset 10, which automatically advances the file pointer to position 15.

- T1 wakes up and calls write("HELLO"): Accidentally writes at offset 15 (instead of offset 0) because T2 altered the shared pointer!

Fix: mutex or pread()

code: mutex

```
void *worker1(void *arg)
{
    pthread_mutex_lock(&mutex);
    // Step 1: T1 wants to move the file pointer to position A (Offset 0)
    lseek(fd, 0, SEEK_SET);

    // Simulate a context switch immediately after seeking
    usleep(100);

    // Step 2: T1 writes data (Expecting to write at Offset 0)
    write(fd, "HELLO", 5);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *worker2(void *arg)
{
    pthread_mutex_lock(&mutex);
    // Step 1: T2 jumps in and moves the file pointer to position B (Offset 10)
    lseek(fd, 10, SEEK_SET);

    // Step 2: T2 writes data
    write(fd, "WORLD", 5);
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

code: pread() = lseek() + write()

```
void *worker1(void *arg)
{
   pwrite(fd, "HELLO", 5, 0);
   return NULL;
}
```

```
void *worker2(void *arg)
{
  pwrite(fd, "WORLD", 5, 10);
  return NULL;
}
```

output: 2 ways is still the same results

```
HELLO□□□□□WORLD
```

7. Condition Variable and Barriers

7.1 Condition Variable

```
In multithreaded programming, a Condition Variable allows one thread to notify other threads about state changes in a shared resource. It also enables waiting threads to automatically go to sleep (block) until they receive a notification.
```

Benefit: Eliminates CPU cycle waste caused by busy-waiting (continuously checking a status flag in a loop).

example:

- Thread 1 (Consumer): Wants to read data $\rightarrow$ Needs to "wait until data is ready.
- "Thread 2 (Producer): Creates data $\rightarrow$ Sets shared_data = 100.
- Workflow: If Consumer runs first -> Data is not ready -> Consumer sleeps. Producer generates data -> Sends a signal -> Consumer wakes up and continues.

API

pthread_cond_t cond = PTHREAD_COND_INITIALIZER : Static initialization

pthread_cond_wait(&cond, &mutex): Atomically unlocks the mutex and puts the thread into the waiting state.

```
Thread
  |
  ▼
lock(mutex)
  |
  ▼
```

```
condition wrong
  |
  ▼
pthread_cond_wait()
  |
  ├── unlock mutex
  |
  └── sleep
       |
       | signal
       ▼
     wake up
       |
       ▼
   lock mutex
       |
       ▼
    continue
```

pthread_cond_signal(&cond): Wakes up at least one thread waiting in the condition variable queue.

```
Producer
   |
   ▼
shared_data = 100
   |
   ▼
pthread_cond_signal()
   |
   ▼
Consumer wake up
```

pthread_cond_broadcast(pthread_cond_t *cond): Wakes up all threads currently sleeping in the condition variable queue.

Condition + Mutex

code:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int data_ready = 0;
void *consumer(void *arg)
{
    pthread_mutex_lock(&mutex);
    while (data_ready == 0)
    {
        printf("Consumer: waiting...\n");

        pthread_cond_wait(&cond, &mutex);
    }
    printf("Consumer: data is ready!\n");
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *producer(void *arg)
{
    sleep(2);
    pthread_mutex_lock(&mutex);
    printf("Producer: preparing data...\n");
    data_ready = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main()
{
    pthread_t consumer_thread;
    pthread_t producer_thread;
    pthread_create( &consumer_thread,  NULL, consumer, NULL);
    pthread_create( &producer_thread,  NULL,  producer,  NULL);
    pthread_join(consumer_thread, NULL);
    pthread_join(producer_thread, NULL);
    return 0;
}
```

output

```
Consumer: waiting...
Producer: preparing data...
Consumer: data is ready!
```

Note:: Always use a while loop instead of an if statement to check the condition state

```
Using if: Check (true) -> Sleep -> Wakeup -> Executes directly (Unsafe).
Using while: Check (true) -> Sleep -> Wakeup -> Re-checks condition (Safe) -> Executes only
when data is valid.
```

7.2 Barrier

Allows a specified number of threads to wait for each other at a synchronization point. No thread can pass the barrier until all participating threads have reached this point, at which time the barrier opens and all threads resume execution simultaneously.

```
Thread 1 ──────────┐                          T1 -> Barrier -> wait
                   │
Thread 2 ──────────┼──→ Barrier               T2 -> Barrier -> wait
                   │
Thread 3 ──────────┘                          T3 -> Barrier -> wait

- when 3 Threads came barrier

Thread 1 ──────────┐
                   │
Thread 2 ──────────┼──→ Barrier  -> all thread continues
                   │
Thread 3 ──────────┘
```

code:

```
pthread_barrier_t barrier;

void *worker(void *arg)
{
   int id = *(int *)arg;
   printf("Worker %d: phase 1\n", id);
```

```
    sleep(id);

    printf("Worker %d: reaching barrier\n", id);

    pthread_barrier_wait(&barrier);

    printf("Worker %d: phase 2\n", id);

    return NULL;
}

int main()
{
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};

    pthread_barrier_init(&barrier, NULL, 3);

    for (int i = 0; i < 3; i++)
    {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < 3; i++)
    {
        pthread_join(threads[i], NULL);
    }

    pthread_barrier_destroy(&barrier);

    return 0;
}
```

output

```
Worker 1: phase 1
Worker 2: phase 1
Worker 3: phase 1
Worker 1: reaching barrier
```

```
Worker 2: reaching barrier
Worker 3: reaching barrier
Worker 3: phase 2
Worker 2: phase 2
Worker 1: phase 2
```

8. Semaphores for synchronization vs. Mutexes for mutual exclusion

A Semaphore is a synchronization tool that operates as an integer and is never allowed to fall below 0. A process can increment or decrement this integer value. If a process attempts to decrement the semaphore value below 0, the kernel blocks that operation until another process increments the semaphore.

API

```
sem_t sem;
sem_init(&sem, 0, n);  => counter = n
sem_wait(&sem);        => counter = counter - 1
sem_post(&sem);        => counter = counter + 1
```

example: T1, T2, T3, T4

```
sem_init(&sem, 0, 3);         -> permit = 3
T1 sem_wait()                 -> Permit = 2
T2 sem_wait()                 -> Permit = 1
T3 sem_wait()                 -> Permit = 0
T4 sem_wait()                    Permit = 0    -> wait  T1 / T2/ T3 sem_post()
```

code:

```
sem_t sem;

void *worker(void *arg)
{
   int id = *(int *)arg;
   printf("Worker %d: waiting for resource\n", id);
   sem_wait(&sem);
   printf("Worker %d: using resource\n", id);
```

```
    sleep(2);
    printf("Worker %d: releasing resource\n", id);
    sem_post(&sem);
    return NULL;
}

int main()
{
    pthread_t threads[4];
    int ids[4] = {1, 2, 3, 4};

    // Có 3 resource
    sem_init(&sem, 0, 3);

    for (int i = 0; i < 4; i++)
    {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < 4; i++)
    {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&sem);

    return 0;
}
```

output

```
Worker 1: waiting for resource              permit = 2
Worker 1: using resource
Worker 2: waiting for resource              permit = 1
Worker 2: using resource
Worker 3: waiting for resource              permit = 0
Worker 3: using resource
Worker 4: waiting for resource              permit = 0

After 2s
```

```
Worker 1: releasing resource                permit = 1
Worker 2: releasing resource                permit = 2
Worker 4: using resource                    permit = 1
Worker 3: releasing resource                permit = 2
Worker 4: releasing resource                permit = 3
```

Note: A Semaphore can also be used like a Mutex if initialized with 1: sem_init(&sem, 0, 1);

8.1 Semaphore trong Producer/Consumer.

sem_t empty; = 5

sem_t full; = 0

Producer: wait(empty) -> put item -> post(full);

Consumer: wait(full) -> take item -> post(empty)

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;  // Index where the Producer writes data
int out = 0; // Index where the Consumer reads data

sem_t empty; // Tracks empty slots (initialized to 5)
sem_t full;  // Tracks filled slots (initialized to 0)

void* producer(void* arg) {
   int item;
   for (int i = 0; i < 8; i++) {
       item = rand() % 100 + 1;
```

```
        // 1. Wait for an empty slot (decrements empty if > 0)
        sem_wait(&empty);

        // --- NO MUTEX NEEDED: Only 1 Producer accesses 'in' ---
        buffer[in] = item;
        printf("[Producer] Put %d at position [%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        // ------------------------------------------------------

        // 2. Signal that a new item is available (increments full)
        sem_post(&full);
        usleep(150000); // Simulate production time (0.15s)
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 8; i++) {
        // 1. Wait for an available item (decrements full if > 0)
        sem_wait(&full);

        // --- NO MUTEX NEEDED: Only 1 Consumer accesses 'out' ---
        item = buffer[out];
        printf("  [Consumer] Took %d from position [%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        // ------------------------------------------------------

        // 2. Signal that a slot has been freed (increments empty)
        sem_post(&empty);
        usleep(300000); // Simulate consumption time (0.3s)
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    // Initialize Semaphores:
    // empty = 5 (all 5 slots are empty initially)
```

```
    // full  = 0 (no items in buffer initially)
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}
```

output

```
[Producer] Đã đưa 84 vào vị trí [0]
  [Consumer] Đã l□y 84 từ vị trí [0]
[Producer] Đã đưa 87 vào vị trí [1]
  [Consumer] Đã l□y 87 từ vị trí [1]
[Producer] Đã đưa 78 vào vị trí [2]
[Producer] Đã đưa 16 vào vị trí [3]
  [Consumer] Đã l□y 78 từ vị trí [2]
[Producer] Đã đưa 94 vào vị trí [4]
[Producer] Đã đưa 36 vào vị trí [0]
  [Consumer] Đã l□y 16 từ vị trí [3]
[Producer] Đã đưa 87 vào vị trí [1]
[Producer] Đã đưa 93 vào vị trí [2]
  [Consumer] Đã l□y 94 từ vị trí [4]
  [Consumer] Đã l□y 36 từ vị trí [0]
  [Consumer] Đã l□y 87 từ vị trí [1]
  [Consumer] Đã l□y 93 từ vị trí [2]
```

9. Lock implementation concepts: atomic instructions, spinlocks vs. blocking locks

(conceptual link to futex)

9.1 Atomic instructions

9.1.1 Atomic

An atomic operation is an operation where other threads cannot see an "in-progress" state. Other threads can only see two states: Not started yet, or Already completed.

Example: You want to open a public restroom door:

```
1. You check if the sign is GREEN (0).


2. If GREEN, you flip the sign to RED (1) to lock the door.


If the two steps (Check + Flip) are separated, two people might look at the GREEN sign at the exact same time and both rush in. The CPU must combine both steps into "ONE SINGLE ACTION" called Check + Modify atomically.
```

9.1.2: CAS (Compare-And-Swap)

an atomic instruction provided by the CPU. It compares the contents of a memory location with a given value (expected), and only if they are the same, modifies the location to a new value (new_value).

idea: CAS(address, expected, new_value)

- address: The memory address to check.
- expected: The value you expect the memory location to hold.
- new_value: The new value you want to write.

```
if (*address == expected)
{
    *address = new_value;
    return success;
}
return failure;
```

Note: The CPU guarantees this action happens in a single instruction at the hardware circuit level—no other thread can interfere in between.

9.1.2 TAS (Test and Set)

An atomic instruction provided by the CPU. It reads the current value of a memory location and sets it to true (1) in a single atomic step, then returns the old value.

Idea: TAS(address)

- address: The memory address to check and set.

```
bool old_value = *address;
*address = true;                  // Always set the flag to locked
return old_value;
```

Why set to true every time?

```
If the lock was unlocked (false), this action immediately locks it for you, and returning
false means you successfully acquired the lock.

If the lock was already locked (true), setting it to true again changes nothing, and returning
true lets you know someone else is inside.
```

9.2 Spinlock

Spinlock is a lock where a thread waiting for the lock continuously loops (spins) in a busy-waiting state until the lock becomes available.

```
while (lock_is_busy)
{
    try_again();
}
```

exmaple: T1 and T2

```
T1 acquires the lock.


T2
│
▼
lock busy?
│
├── YES → check again
│         check again
│         check again
│         ...
```

```
|
└── NO → acquire
```

Pro and Con

```
Pro: Extremely fast for short waiting times because it avoids Context Switch overhead.


Con: Wastes CPU cycles while waiting.
```

9.3: Blocking lock

Blocking Lock (Futex) puts waiting threads to sleep instead of spinning, dropping CPU usage to 0%.

9.4 Compare Spinlock and Blockinglock

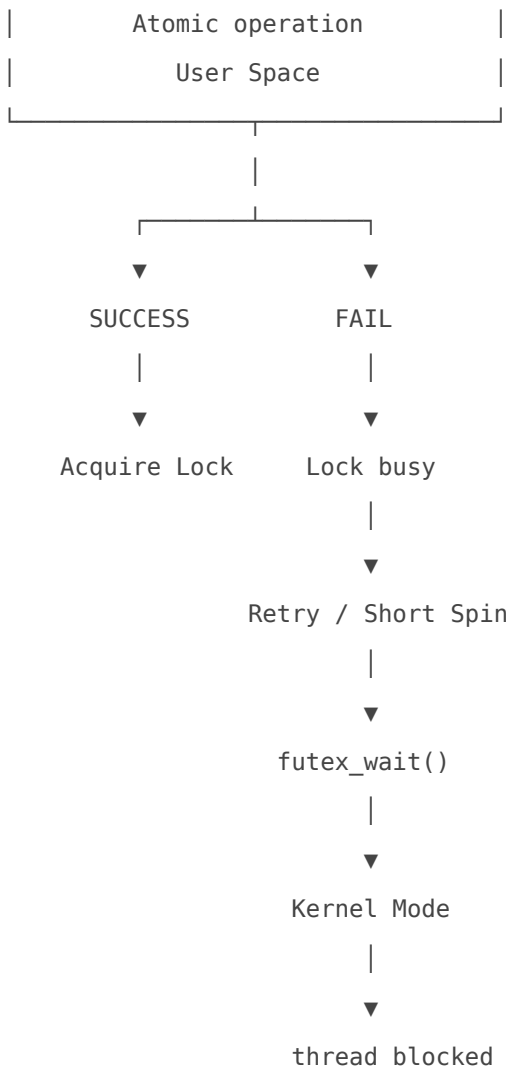
Feature	Spinlock	Blocking Lock (Mutex/Futex)
Waiting behavior	Keeps checking in a loop (**Busy-waiting**).	Asks the OS to put the thread to **Sleep**.
Thread state	Remains **RUNNING**.	Changes to **WAITING / SLEEPING**.
CPU usage	**Very high** (~100% of 1 CPU core while waiting).	**Zero** (0% CPU core while thread is sleeping).
Context Switch cost	**None** (Thread stays on the CPU).	**Yes** (Cost of saving/loading registers and clearing cache).
Response delay (Latency)	**Extremely low** (Gets lock immediately when freed).	**Higher** (Must wait for the OS to wake the thread up).
Best use case	**Very short** lock hold time. Great for Drivers, Kernel.	**Long** lock hold time (File I/O, Network, heavy work).

9.5 Conceptual Link to Futex

Uncontended Case (Fast Path – Spinlock-like mechanism): The thread attempts to acquire the lock entirely in Userspace (without invoking the Kernel) using CPU atomic operations (e.g., Compare-And-Swap). This operation is extremely fast.

Contended Case (Slow Path – Blocking Lock mechanism): If the resource is detected to be held by another thread, Futex issues a System Call to the Kernel to put the thread into a wait queue and send it to "sleep".

```
              [ Try to Acquire Lock ]
                        |
                        ▼
          ┌──────────── FAST PATH ────────────┐
```



10. Deadlock: the four necessary conditions, prevention and avoidance strategies (e.g. lock ordering), detecting and breaking a deadlock

Deadlock: A situation where threads/processes wait for each other indefinitely, so none can proceed.

Example:

- T1 holds Lock A, waits for Lock B.
- T2 holds Lock B, waits for Lock A.

=> Circular wait (T1 waits for T2, T2 waits for T1) → Deadlock!

10.1 the four necessary conditions

1. **Mutual Exclusion:** Resources cannot be shared. Only one thread can use the resource at a time.
   - *Example:* T1 holds Lock A, so T2 cannot acquire it.
2. **Hold and Wait:** A thread holding at least one resource is actively waiting to acquire additional resources held by others.
   - *Example:* T1 holds Lock A and waits for Lock B, without releasing A.
3. **No Preemption:** Resources cannot be forcibly taken away by the OS. They are released only voluntarily by the holding thread.
   - *Example:* Kernel cannot force T1 to release Lock A for T2; T2 must wait until T1 calls `unlock()`.
4. **Circular Wait:** A closed chain of threads exists such that each thread waits for a resource held by the next.
   - *Example:* T1 waits for T2 → T2 waits for T3 → T3 waits for T1.

Note:

- For a deadlock to occur, **all 4 conditions** must hold true simultaneously.
- Breaking even **one** of these four conditions is sufficient to prevent deadlocks completely.

10.2 Prevention

- **Break Mutual Exclusion:** Use lock-free data structures (Atomic operations) or read-only sharing.
- **Break Hold and Wait:** Force threads to acquire all required locks at once, or release all current locks before requesting new ones.
- **Break No Preemption:** Use `try_lock()` with timeouts; if a thread fails to acquire a lock, it must yield its existing locks.
- **Break Circular Wait (Most Common):** Enforce a strict global **Lock Ordering** (e.g., always acquire Lock A before Lock B).

10.3 Avoidance strategies (lock ordering)

10.3 Avoidance Strategies (Lock Ordering)

Always get locks in the **same fixed order** for all threads to stop deadlocks.

- **Rule:** Give each lock a number (e.g., Lock 1 < Lock 2). Every thread must get Lock 1 before Lock 2.
- Breaks Condition 4 (**Circular Wait**). Threads will never wait for each other in a circle.

example:

T1: A -> B

T2: A -> B

Thread 1:

```
pthread_mutex_lock(&lockA);
pthread_mutex_lock(&lockB);
```

Thread 2

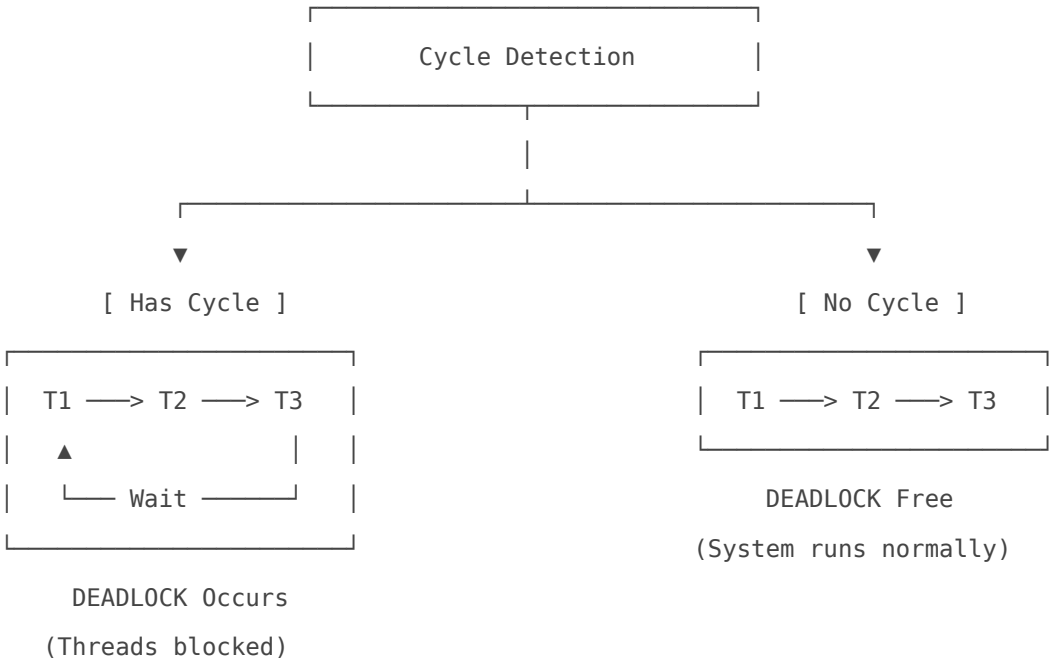
```
pthread_mutex_lock(&lockA);
pthread_mutex_lock(&lockB);
```

Cannot be used

```
pthread_mutex_lock(&lockB);
pthread_mutex_lock(&lockA);
```

10.4 Detection

A **Wait-For Graph (WFG)** is a directed graph that shows which thread is waiting for which thread.



10.5 Breaking deadlock

If a deadlock happens, the system can break it by:

1. **Release Resource (Preemption):** Force a thread to release its lock.
2. **Abort Thread/Process:** Terminate one or more threads in the cycle to break the loop.

3. **Rollback:** Revert thread states back to a safe checkpoint (common in Database Transactions).

Key Takeaway: In practice, **prevention** (e.g., Lock Ordering) is much easier and cheaper than **recovery**.

Studio / Lab

Lab1: Create and join multiple threads sharing a counter; observe a race condition on the unprotected shared data

2.1 Share data

Lab2: Fix the race condition using a mutex lock; verify correctness under load

3.4 Fixing the Race Condition Program

Lab3: Test the reentrancy (thread-safety) of a self-written function and fix it if it is not reentrant

code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
char *get_message(const char *name)
{
   static char buffer[100];
   snprintf(buffer, sizeof(buffer), "Hello %s", name);
   return buffer;
}
void *worker(void *arg)
{
   char *name = (char *)arg;
   char *msg = get_message(name);
   usleep(1000);
   printf("%s\n", msg);
   return NULL;
}
```

```
int main()
{
   pthread_t t1, t2;
   char *name1 = "Worker1";
   char *name2 = "Worker2";
   pthread_create(&t1, NULL, worker, name1);
   pthread_create(&t2, NULL, worker, name2);
   pthread_join(t1, NULL);
   pthread_join(t2, NULL);
   return 0;
}
```

output

```
Hello Worker2
Hello Worker2
```

explain : Race Condition Example: Static Buffer

- **Problem:** `static char buffer[100]` is stored in the **Data Segment** (shared memory), not on the thread's stack.
- **Issue:** Thread 2 overwrites `buffer` while Thread 1 is sleeping (`usleep`). Since both threads point to the same memory address, both print `"Hello Worker2"`.

=> Functions using internal `static` state are **Not Thread-Safe**.

Fix

Method 1: Caller-Allocated Buffer (Stack Memory)

```
void get_message(const char *name, char *buffer, size_t size)
{
   snprintf(buffer, size, "Hello %s", name);
}

void *worker(void *arg)
{
   char *name = (char *)arg;
   char msg[100]; // This array is allocated on the current thread's private stack
   get_message(name, msg, sizeof(msg));
   usleep(1000);
   printf("%s\n", msg);
```

```
    return NULL;
}
```

output

```
Hello Worker1
Hello Worker2
```

Method 2: Dynamic Heap Allocation (Caller Frees)

```
char *get_message(const char *name)
{
  char *buffer = (char *)malloc(100);
  if(buffer == NULL) return NULL;
  snprintf(buffer, 100, "Hello %s", name);
  return buffer;
}

void *worker(void *arg)
{
  char *name = (char *)arg;
  char *msg = get_message(name);
  usleep(1000);
  if(msg){
       printf("%s\n", msg);
       free(msg);
  }
  return NULL;
}
```

output

```
Hello Worker1
Hello Worker2
```

Lab 4 Implement a producer-consumer program using a mutex and a condition variable

code:

```
#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];

int count = 0;
int in = 0;
int out = 0;


pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg)
{
   for (int i = 1; i <= NUM_ITEMS; i++)
   {
       pthread_mutex_lock(&mutex);

       // Wait while buffer is full
       while (count == BUFFER_SIZE)
       {
           pthread_cond_wait(&not_full, &mutex);
       }

       // Produce item
       buffer[in] = i;

       printf("Producer produced: %d\n", i);

       in = (in + 1) % BUFFER_SIZE;
       count++;

       // Tell consumer that buffer is not empty
       pthread_cond_signal(&not_empty);

       pthread_mutex_unlock(&mutex);
```

```
        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= NUM_ITEMS; i++)
    {
        pthread_mutex_lock(&mutex);

        // Wait while buffer is empty
        while (count == 0)
        {
            pthread_cond_wait(&not_empty, &mutex);
        }

        // Consume item
        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        count--;

        // Tell producer that buffer is not full
        pthread_cond_signal(&not_full);

        pthread_mutex_unlock(&mutex);
 usleep(150000);
    }

    return NULL;
}


int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;
```

```
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread,NULL,consumer,NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}
```

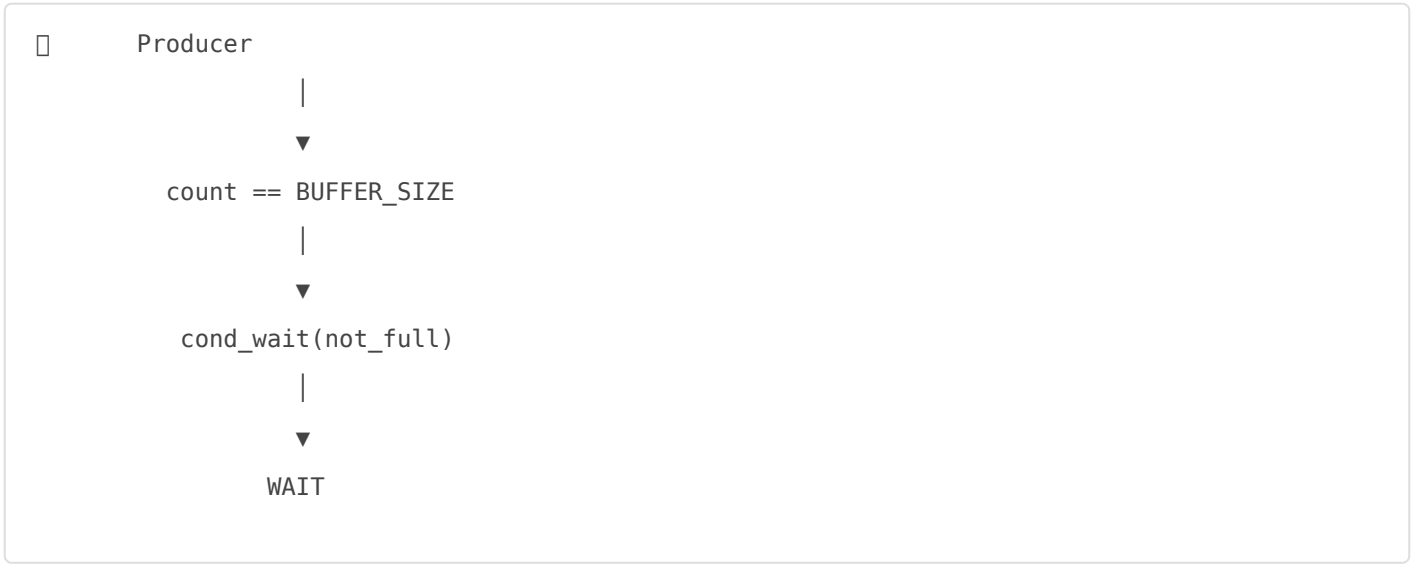
output

```
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
```

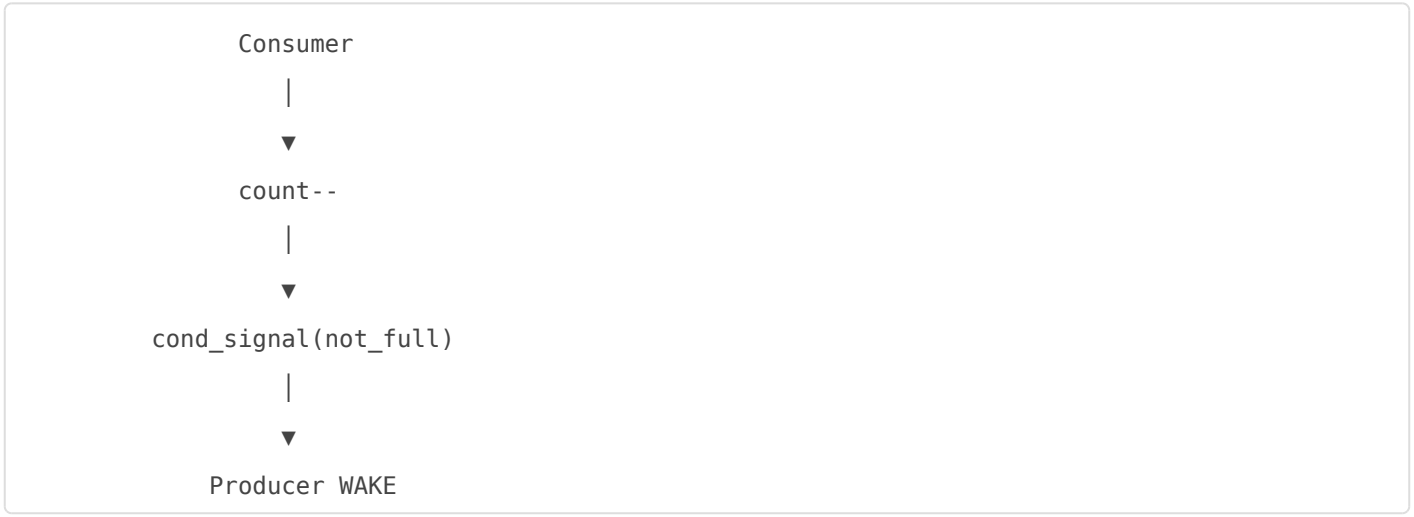
explain:



if buffer full



Consumer take item



if buffer empty



```
              WAIT
```

Producer add item:

```
            Producer
               |
               ▼
            count++
               |
               ▼
      cond_signal(not_empty)
               |
               ▼
        Consumer WAKE
```

Lab 5: Reproduce a deadlock between two threads acquiring two locks in opposite order, then fix it via consistent lock ordering

code:

T1: A → B

T2: B → A

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lockB = PTHREAD_MUTEX_INITIALIZER;

void *worker1(void *arg)
{
    printf("Worker 1: locking A\n");

    pthread_mutex_lock(&lockA);

    printf("Worker 1: got A\n");
```

```
    sleep(1);

    printf("Worker 1: locking B\n");

    pthread_mutex_lock(&lockB);

    printf("Worker 1: got B\n");

    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);

    return NULL;
}

void *worker2(void *arg)
{
    printf("Worker 2: locking B\n");

    pthread_mutex_lock(&lockB);

    printf("Worker 2: got B\n");

    sleep(1);

    printf("Worker 2: locking A\n");

    pthread_mutex_lock(&lockA);

    printf("Worker 2: got A\n");

    pthread_mutex_unlock(&lockA);
    pthread_mutex_unlock(&lockB);

    return NULL;
}

int main()
{
    pthread_t t1, t2;
```

```
    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Main: done\n");

    return 0;
}
```

output:

```
terminal1:

Worker 1: locking A
Worker 1: got A
Worker 2: locking B
Worker 2: got B
Worker 1: locking B
Worker 2: locking A

terminal 2:

app(10821)─┬─{app}(10822)    T1 deadlock
           └─{app}(10823)    T2 deadlock
```

Fix

T1: A -> B

T2: A -> B

```
pthread_mutex_t lockA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lockB = PTHREAD_MUTEX_INITIALIZER;

void *worker1(void *arg)
{
```

```
    printf("Worker 1: locking A\n");

    pthread_mutex_lock(&lockA);

    printf("Worker 1: got A\n");

    sleep(1);

    printf("Worker 1: locking B\n");

    pthread_mutex_lock(&lockB);

    printf("Worker 1: got B\n");

    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);

    return NULL;
}

void *worker2(void *arg)
{
   printf("Worker 2: locking A\n");

   pthread_mutex_lock(&lockA);

   printf("Worker 2: got A\n");

   sleep(1);

   printf("Worker 2: locking B\n");

   pthread_mutex_lock(&lockB);

   printf("Worker 2: got B\n");

   pthread_mutex_unlock(&lockB);
   pthread_mutex_unlock(&lockA);

   return NULL;
```

```
}


int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker1, NULL);
    pthread_create(&t2, NULL, worker2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Main: done\n");

    return 0;
}
```

output

```
Worker 1: locking A
Worker 1: got A
Worker 2: locking A
Worker 1: locking B
Worker 1: got B
Worker 2: got A
Worker 2: locking B
Worker 2: got B
Main: done
```

Revision #5
Created 2026-08-28 09:20:09 UTC by Dat Nguyen
Updated 2026-09-14 01:55:26 UTC by Dat Nguyen